Algorithm Desigh - Exercise Sheet
Backtracking and Branch & Bound

May 4, 2026

1. The number of recursive calls of the function `bactracking(int level)` for $n = 4$ is 17, according to the implementation in the lecture. In addition to the 16 nodes represented in the recursive call tree in which each node is a 4x4 chessboard, there is also the initial call, the root of the tree, that is, `bactracking(1)`. Thus, validating the partial solution reduces the number of calls from 341, that is the number of calls done by an exhaustive search that tests the restriction of attack positions only at the end; to just 17. How many recursive calls do the two methods make for $n = 3$? How do you expect the ratio of the number of calls generated by backtracking relative to exhaustive search to change for $n = 5, 6, \ldots$ and how do you interpret this?

2. The number of nodes in the exhaustive traversal tree built to determine the satisfiability of the formula f below is $2^4 - 1$. How many of these nodes are visited in a backtracking procedure that eliminates partial solutions that cannot lead to satisfiability?

   $f = (\neg x_1 \vee x_2) \wedge (x_2 \vee \neg x_3) \wedge (\neg x_3 \vee x_3)$

3. Design a backtracking algorithm for the subset sum problem. The design implies specifying the following information:

   (a) how a solution is represented

   (b) what are the partial solutions

   (c) which are the direct successors of a partial solution

   (d) when a partial solution is viable

4. Design a backtracking algorithm for the Sudoku problem.

5. Design a branch & bound algorithm for the Maximum Independent Set problem. Propose a non-trivial estimation function for *maxRest* and argue that it is correct.